

2-Análise Léxica e Expressões Regulares

Luiz Faria

Constantino Martins

Funções da Análise Léxica

... a [i n d] = 1 0 * x 1 + y 1 ...

Sequência de *tokens*:

... a [ind] = 10 * x1 + y1 ...

- Principal: Agrupar os caracteres individuais do programa fonte em “palavras”, designadas por *tokens*, que são as unidades básicas das linguagens de programação, e que serão usados pelo analisador sintáctico (*parser*)
- Outras:
 - ◆ ignorar alguns tipos de caracteres (espaços, *tabs*, LF, CR) e os comentários
 - ◆ produzir mensagens de erro quando algum caracter ou sequência de caracteres não são reconhecidos
 - ◆ localizar os *tokens* no texto fonte (nº de linha, nº de coluna)
 - ◆ actuar como pré-processador (em certos compiladores)

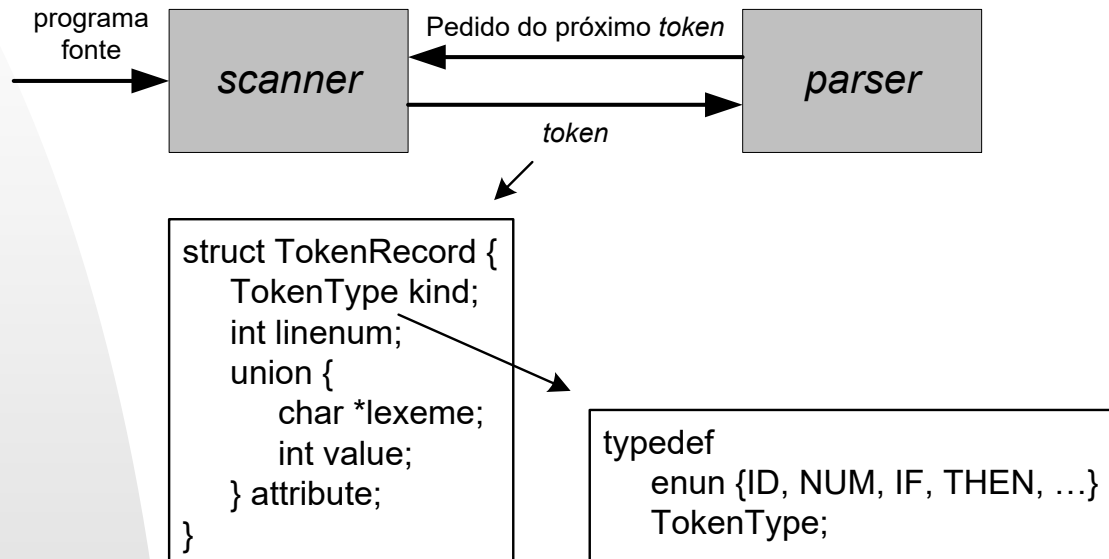
Tokens e Lexemas

- *Token* – representa um conjunto de cadeias de entrada possível
- *Lexema* – é uma determinada cadeia de entrada associada a um *token*

<i>Token</i>	Alguns lexemas possíveis
for	for
if	if
while	while
número	1089, 142857, .0, 3.14159
identificador	i, j, contador, nome_aluno
abr_par	(

Actuação do Analisador léxico

O analisador léxico (*scanner*) actua como uma “subrotina” do analisador sintáctico (*parser*):



Especificação do analisador léxico

- Definição do conjunto de *tokens*
 - ◆ Descrição genérica (através das suas propriedades) das entidades constituintes da linguagem fonte
 - ◆ Dependente da gramática da linguagem fonte
- Exemplos de algumas dessas entidades:
 - ◆ Palavras-chave da linguagem (p. ex: if then while do, etc.)
 - ◆ identificadores (de variáveis, procedimentos, tipos, campos de registos, etc.)
 - ◆ operadores (aritméticos, lógicos, booleanos, etc.)
 - ◆ constantes (também chamadas literais - inteiros, reais, caracteres, strings, etc.)
 - ◆ sinais de pontuação (; , : [(.. etc.)
- Definição das regras de formação dos *tokens*
 - ◆ Exemplo: identificador - sequência de letras ou dígitos, iniciada por letra

Especificação dos *tokens*

- Os *tokens* são geralmente especificados utilizando expressões regulares
- As expressões regulares são regras bem definidas que geram um conjunto de sequências de caracteres (*strings*)
- O conjunto de *strings* gerado por uma expressão regular chama-se uma linguagem regular
- As strings geradas por uma expressão regular são construídas com caracteres de um conjunto de símbolos chamado alfabeto (Σ)
- A linguagem regular gerada por uma expressão regular 'r' nota-se como $L(r)$ (conjunto de *strings* que são sequências de símbolos de Σ)
- A linguagem regular vazia (sem qualquer string) nota-se por Φ .
- A linguagem vazia (Φ) é diferente da linguagem que só contém a *string* vazia (que se nota por ϵ) $L(\epsilon) = \{ \epsilon \}$

Expressões regulares básicas

- As expressões regulares são construídas efectuando operações sobre as chamadas expressões regulares básicas
- Uma expressão regular básica num alfabeto Σ pode ser:
 - ◆ Um único carácter do alfabeto Σ - estas expressões regulares especificam-se escrevendo apenas o próprio carácter
(ex: $\Sigma = \{a, b, c\}$ $r = b$ $L(r) = \{ b \}$)
 - ◆ A *string* vazia ε
(ex: se a expressão regular básica for $r = \varepsilon$ então $L(r) = \{\varepsilon\}$)
 - ◆ A linguagem vazia Φ
(ex: se $r = \Phi$ então $L(r) = \{ \}$)

Operações sobre expressões regulares

- São quatro as operações sobre expressões regulares que fazem parte da sua definição formal (em ordem crescente de prioridade):
 - ◆ Alternativa / união: Uma expressão regular da forma $s \mid t$, onde s e t são expressões regulares; se $r = s \mid t$ então $L(r) = L(s) \cup L(t)$
 - ◆ Concatenação: Uma expressão regular da forma st , onde s e t são expressões regulares; se $r = st$ então $L(r) = L(s)L(t)$
 - ◆ Fecho de Kleene / repetição: Uma expressão regular da forma s^* , onde s é uma expressão regular; se $r = s^*$ então $L(r) = L(s)^* = \bigcup_{n=0, \infty} L(s)^n$
 - ★ r^* fecho de r $\Rightarrow$ conjunto de todas as cadeias de símbolos formadas a partir de r , incluindo a cadeia vazia ϵ
 - ★ $r^+ \Rightarrow r^* - \{\epsilon\}$
 - ◆ Uso de parêntesis: Uma expressão regular da forma (s) , onde s é uma expressão regular; se $r = (s)$ então $L(r) = L(s)$; os parêntesis servem apenas para modificar a precedência das operações
- Notar o uso dos caracteres $\mid$, $*$, $($ e $)$ para designar as operações entre expressões regulares: estes caracteres com significado especial designam-se por metacaracteres. Se estes caracteres também pertencerem ao alfabeto Σ então o seu uso como caracteres deve ser distinguido de alguma forma (p. ex. colocando-os entre plicas - $'(')$

Exemplos

Concatenação entre conjuntos de *strings*:

Suponhamos que $S = \{aa, b\}$ e $T = \{a, bb\}$
então $R = ST = \{aaa, aabb, ba, bbb\}$

Fecho de Kleene de um conjunto de *strings*:

Suponhamos que $S = \{a\}$
então $S^* = S^0 \cup S^1 \cup S^2 \cup S^3 \cup \dots$
onde $S^0 = \{\epsilon\}$, $S^1 = S$, $S^2 = SS$, $S^3 = SSS$, ...
logo $S^* = \{\epsilon, a, aa, aaa, aaaa, \dots\}$

No alfabeto $\Sigma = \{a, b, c\}$ escrever uma expressão regular que represente todas as *strings* que contêm um só b:

$$(a \mid c)^* b (a \mid c)^*$$

A expressão de cima gera uma linguagem que contém por exemplo:

b, abc, abaca, ccbaca, cccccb, etc.

No alfabeto $\Sigma = \{a, b, c\}$ escrever uma expressão regular que represente todas as *strings* que contêm no máximo um b:

$$(a \mid c)^* \mid (a \mid c)^* b (a \mid c)^*$$

ou

$$(a \mid c)^* (b \mid \epsilon) (a \mid c)^*$$

Ambas as expressões de cima geram a mesma linguagem que é a que foi especificada por palavras

(nota: o operador $\mid$ é aquele que apresenta menor prioridade)

Mais exemplos

No alfabeto $\Sigma = \{a, b, c\}$ escrever uma expressão regular que represente todos as *strings* que não contêm b's consecutivos.

Vamos por partes:

Strings que não contêm b's: $(a \mid c)^*$

Strings que têm qualquer coisa a seguir a um b:

$(b(a \mid c))^*$

Combinando os 2 anteriores temos realmente *strings* sem b's consecutivos:

$((a \mid c)^* \mid (b(a \mid c))^*)^* = ((a \mid c) \mid b(a \mid c))^*$

uma vez que $(r^* \mid s^*)^* = (r \mid s)^*$

No entanto esta ainda não é a solução para o problema inicial, uma vez que não contempla os *strings* que terminam em b.

Para chegar à solução geral basta agora dar essa possibilidade:

$((a \mid c) \mid b(a \mid c))^* (b \mid \epsilon)$

No alfabeto $\Sigma = \{a, b, c\}$, *strings* com um n^o par de a's:

$((b \mid c)^* a (b \mid c)^* a)^* (b \mid c)^*$

Um exemplo prático $\rightarrow$ a definição dos números inteiros no alfabeto $\Sigma = \{-, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$\text{num} = 0 \mid ('-' \mid \epsilon) [1-9] [0-9]^*$

$[x-y]$ é uma abreviatura para a alternativa $(x \mid \dots \mid y)$ de todos os símbolos do alfabeto compreendidos entre x e y, considerando o alfabeto um conjunto ordenado.

Abreviaturas em expressões regulares

- Uma ou mais repetições: $r r^*$ pode abreviar-se para r^+
- Zero ou uma instância: $(r \mid \varepsilon)$ pode abreviar-se para $r?$
- Qualquer carácter do alfabeto: $(c_1 \mid c_2 \mid \dots \mid c_n)$ para todo o alfabeto, pode abreviar-se por $.$
- Alternativas entre alguns caracteres ou numa gama de caracteres considerando o alfabeto um conjunto ordenado:
 - $(a \mid b \mid c)$ pode abreviar-se por $[abc]$
 - letra = $[a-zA-Z]$
 - char_identificador = $[a-zA-Z_]$
- Qualquer carácter do alfabeto excepto alguns:
 - alfabeto a considerar: todas as letras minúsculas
 - (qualquer símbolo excepto $a|b|c$) = $[d-z]$ pode abreviar-se por $\sim(a \mid b \mid c)$ ou mais simplesmente por $\sim[abc]$ ou ainda abc
- Subexpressões:
 - natural = $[0-9]^+$
 - natural_com_sinal = $(+ \mid -)?$ natural

Expressões regulares na definição de *tokens*

- Para a construção de um *scanner* deveremos começar por escrever uma expressão regular que compreenda a definição de todos os *tokens* da linguagem.
- Processo:
 1. Escrever uma expressão regular para cada classe de *token* da linguagem
 - r1 = palavra_chave = if | then | while | do | ...
 - r2 = identificador = letra (letra | dígito)*
 - r3 = inteiro = dígito⁺
 - r4 = espaço_em_branco = (\n | \t | ' ' | comentario)⁺
 - ...
 - (esta última definição não é um *token*, mas também tem de ser reconhecida)
 2. Escrever uma expressão regular global (R) agrupando todas as definições anteriores:
 - R = palavra_chave | identificador | inteiro | espaço_em_branco | ...

Algoritmo para reconhecimento de *tokens*

1. Considere-se a entrada $x_1 x_2 \dots x_n$ do programa fonte:
Determinar o maior i tal que $1 \leq i \leq n$ e $x_1 \dots x_i \in L(R)$
2. Se $x_1 \dots x_i \in L(R)$ então $x_1 \dots x_i \in L(r_k)$ para um ou mais k 's
O menor k para o qual se verifica a condição anterior define o *token* ao qual pertence a *string* $x_1 \dots x_i$
3. Remover $x_1 \dots x_i$ da entrada e voltar ao passo 1.

Notas:

- ◆ O passo 2 dá prioridade às classes de *tokens* definidas em 1º lugar; p. ex., 'then' é reconhecido como palavra chave e não como identificador
- ◆ Se verificarmos que $x_1 \dots x_i \in L(R)$ e $x_1 \dots x_j \in L(R)$ devemos considerar a *string* mais longa. Permite reconhecer, p. ex., *tokens* como '>=' onde '>' também é *token*. → (regra da *substring* mais longa ou princípio do *maximal munch*)
- ◆ Erros: e se nenhum $x_1 \dots x_i \in L(R)$?
Escrever uma regra (a última) (r_j) que abranja todas as *strings* "erradas" (*catch all*) ou descartar x_1 e voltar ao passo 1.

Limitações das ER's

- Certos *tokens* são difíceis de especificar numa expressão regular, como os comentários delimitados por 2 caracteres, em ordem inversa
Ex: comentários em C - delimitados por /* e */ :
Uma solução: /*' /* ('*' [^/ ']* /*)* '*/
- Há certos *tokens* que não são especificáveis com expressões regulares
Ex: Os comentários em Modula-2 podem ser imbricados:
(* isto é um comentário em (* Modula - 2 *) *)
- Certas linguagens exigem que se conheçam vários caracteres para além do fim do *token* que se está a reconhecer (*lookahead*)
Ex: No FORTRAN as palavras chave não são reservadas nem o espaço em branco é significativo:
IF (I F .EQ. 0) THENTHEN=1.0 (instruções válidas em FORTRAN)
DO99I=1,10
DO99I=1.10

Nestes casos é necessário usar um *scanner* não totalmente baseado em expressões regulares → gramáticas independentes do contexto (tipo 2)